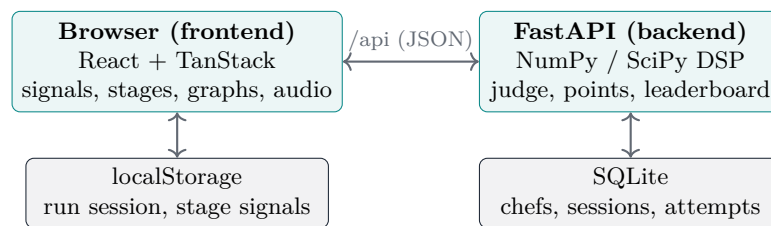

WaveKitchen / WaveBakery

Detailed Architecture

The inner mathematics, the function behind every operation, and the step-by-step workings of generation, filtering, mixing, seasoning, marinating, cooking, the oven, the z-plane, scoring and the leaderboard



Contents

1	System architecture	3
1.1	Processes and responsibilities	3
1.2	Frontend storage (<code>localStorage</code>)	3
1.3	Backend tables (<code>backend/app/models.py</code>)	4
2	Conventions: how a signal is represented	4
2.1	The <code>PipelineSignal</code>	4
2.2	From samples to sound (<code>frontend/src/lib/audio.ts</code>)	4
3	Signal generation	5
3.1	Frontend ingredient models (<code>frontend/src/lib/signals.ts</code>)	5
3.2	Backend ingredient synthesis and contamination	5
4	Filtering	6
4.1	Server filter (authoritative)	6
4.2	Cleaning (prep) score — <code>metrics.prep_quality</code>	6
4.3	Frontend Filtering Lab (<code>routes/filtering.tsx</code>)	6
5	Mixing	7
6	Seasoning and marinating	7
6.1	Seasoning — <code>computeSeasonedSignal</code>	7
6.2	Marinating — <code>computeMarinatedSignal</code>	7
7	Cooking — convolution	8
7.1	Impulse responses	8
7.2	<code>computeConvolvedSignal</code> (<code>lib/pipeline.ts</code>)	8
8	Precision Oven (<code>frontend/src/lib/precision-oven-dsp.ts</code>)	8
8.1	The problem: a burnt overtone (<code>frontend/src/lib/delivery.ts</code>)	8
8.2	Maximum frequency and the Nyquist rate	9
8.3	Sampling and aliasing demonstration — <code>analyzeSamplingAndAliasing</code>	9
8.4	Spectrum used for tuning — <code>computeDiscreteSignalFFT</code>	9
8.5	Equaliser — <code>tuneOvenFrequencies</code>	9
8.6	Reconstruction — <code>computeSignalIFFT</code>	9
8.7	Oven score	10
9	System Delivery — the z-plane (<code>frontend/src/lib/z-system.ts</code>)	10
10	Similarity metrics	11
10.1	Frontend (<code>computeSignalSimilarity</code> , <code>frontend/src/lib/dsp.ts</code>)	11
10.2	Server dish score (<code>metrics.dish_metrics</code>)	11

11 Scoring, step by step	12
11.1 During the run: stage accuracies	12
11.2 The reference dish	12
11.3 Server judgement — <code>gameplay.judge</code>	12
11.4 Submission (exactly once)	13
11.5 What the score screen shows (<code>routes/score.tsx</code>)	13
12 Points, ranks, tiers and leaderboards	13
12.1 Career points and ranks (<code>backend/app/gameplay.py</code>)	13
12.2 Server leaderboards (<code>backend/app/routers/leaderboard.py</code>)	13
12.3 Leaderboard page (<code>frontend/src/routes/leaderboard.tsx</code>)	14
12.4 On-device leaderboard (<code>frontend/src/lib/leaderboard.ts</code>)	14
13 Timer, pause and sound	14
14 Signal Playground (<code>frontend/src/lib/playground.ts</code>)	14
15 Security and validation	15
16 Function index	15

1 System architecture

1.1 Processes and responsibilities

Part	Responsibility
<code>run.py</code>	Starts the backend (<code>uvicorn app.main:app --reload</code> , port 8000; on Windows in its own hidden console so its reload Ctrl+C cannot stop the runner) and the Vite dev server (port 5173). Relays backend logs; stops both on exit.
Frontend	Synthesises ingredient signals, runs every station's DSP, draws, plays the signals and the sound effects, runs the timer (with pause and the start countdown), keeps the run in <code>localStorage</code> , and talks to the backend through <code>frontend/src/lib/api.ts</code> (header <code>X-Player-Token</code>).
Backend	Owns chefs (hashed passwords, tokens), sessions and attempts; regenerates ingredients from a seed; rebuilds and scores the dish; computes points, ranks, tier unlocks and leaderboards.

1.2 Frontend storage (`localStorage`)

Key	Contents
<code>wavebakery_recipe_session</code>	The run: recipe, difficulty, start/end time, every stage accuracy, every dial (seasoning, marinate, appliance, bowl, oven, cart), final score/stars, backend session id and result, and the pause state (paused flag, time left when paused, countdown pending). Type <code>RecipeRunSession</code> in <code>lib/recipes.ts</code> .
<code>wavebakery_progress_<recipe></code>	Highest unlocked level (1–9).
<code>wavebakery_pipeline_<recipe>_<stage></code>	A saved <code>PipelineSignal</code> for stage <code>raw</code> , <code>filtered</code> , <code>mixed</code> , <code>seasoned</code> , <code>marinated</code> , <code>cooked</code> , <code>delivered</code> .
<code>wavebakery_filtered_ingredients_<recipe></code>	Filtered samples per washable ingredient.
<code>wavebakery_cooked_signal_<recipe></code>	The cooked dish (<code>CookedSignalData</code>).
<code>wavebakery_selected_ingredients_<recipe></code>	Ingredients picked in Generate.
<code>wavebakery_leaderboard_entries</code>	The on-device leaderboard.
<code>wavekitchen_player_token</code> , <code>wavebakery_chef_name</code>	Server token and chef name.
<code>wavebakery_recipe_best_scores</code>	Best total per recipe.
<code>wavebakery_sound_settings</code>	Sound on/off and volume (0–100), set in Settings or the pause menu (<code>lib/sound.ts</code>).

Changing a stage calls `invalidateDownstreamStages` (`lib/pipeline.ts`), which deletes every later stage's saved signal so nothing stale is carried forward.

1.3 Backend tables (backend/app/models.py)

Table	Fields that matter
players	handle, token, script password_hash, points, unlocked tier.
ingredients, appliances, recipes	Catalogue from seed.py; recipes hold targets (seasoning, blend, marinate, appliances), tier, tolerance, noise difficulty. A recipe removed from the book stays in the table (old runs reference it) but is retired (is_active false).
game_sessions	One run: seed, status (active/served/abandoned), difficulty, params (JSON).
session_ ingredients	Per-slot seed, contaminants, filter bands/tools, prep score, accepted.
attempts	A served dish: score, stars, prep score, SNR, MSE, correlation, spectral similarity, overall score and time bonus, points, difficulty, scoring version, notes. The leaderboard's source.

2 Conventions: how a signal is represented

2.1 The PipelineSignal

Every frontend stage produces

$$\mathbf{x} = (x[0], \dots, x[N-1]), \quad N = 401, \quad t_n = \frac{n}{N-1} \in [0, 1],$$

plus metadata (frequency label, duration label, and optionally the 2-D mixing curve). A frequency of f “game Hz” means f cycles in the window.

Helper functions (frontend/src/lib/pipeline.ts)

- **sampleAt**($\mathbf{x}, u$): periodic linear interpolation. With $u' = u \bmod 1$, $p = u'(N-1)$, $i = \lfloor p \rfloor$: $x(u) = x[i] + (p-i)(x[i+1] - x[i])$.
- **resampleSignal**($\mathbf{x}, M$): $y[m] = \text{sampleAt}(\mathbf{x}, m/(M-1))$.
- **normalizeSamples**($\mathbf{x}, 0.95$): scales *down only*: $y = x \cdot \min(1, 0.95/\max_n |x[n]|)$.

2.2 From samples to sound (frontend/src/lib/audio.ts)

SignalAudioPlayer treats the window as one period-group and loops it. With the signal's frequency label f (default 4),

$$f_0 = \text{clip}\left(220 \cdot \frac{f}{4}, 110, 880\right) \text{ Hz}, \quad \text{windows per second} = \frac{f_0}{\max(1, f)}.$$

Output sample k at the device rate F_s ($t = k/F_s$) is $s(t) = x((t \cdot f_0/f) \bmod 1) \cdot g \cdot e(t)$ with gain $g = 0.75/\max |x|$ and a 30 ms attack / 120 ms release envelope $e(t)$. So the audible fundamental is f_0 and the timbre is exactly the drawn waveform. The recorded chicken plays its decoded WAV buffer directly.

Loop seam (**smoothLoopSeam**): looping jumps from $x[N-1]$ back to $x[0]$. If that jump is larger than $1.5\times$ the largest step inside the signal (a signal that does not complete whole cycles in the window, e.g. carrot, milk or a delayed dish), the last 5% of the window is blended towards $x[0]$ with

a raised-cosine ramp $x[n] + (x[0] - x[N - 1])\frac{1}{2}(1 - \cos \pi u)$, removing the click that would otherwise repeat at the loop rate. Genuine edges (square waves) are left untouched.

3 Signal generation

3.1 Frontend ingredient models (`frontend/src/lib/signals.ts`)

Each ingredient is an `IngredientSignalDefinition` with `evaluate`, `generateSamples` and, for 2-D shapes, a `parametricCurve`. The router `getRecipeIngredientSamples` (`lib/pipeline.ts`) returns 401 samples:

1. **Chicken** $\rightarrow$ `generateChickenSamples`: linear interpolation of the decoded PCM (`lib/chicken-samples.ts`) over the whole 2.16 s clip.
2. **Catalogued ingredient** $\rightarrow$ its `generateSamples( $f, A, \phi$ , noise)`.
3. **Fallback**: $x[n] = A \sin(2\pi f t_n + \phi)$.

Periodic families

$$\begin{aligned} \text{triangle (Cheese): } x(t) &= A \frac{2}{\pi} \arcsin(\sin(2\pi f t + \phi)) = \frac{8A}{\pi^2} \sum_{k \text{ odd}} \frac{(-1)^{(k-1)/2}}{k^2} \sin(2\pi k f t + \dots) \\ \text{square (Sugar, Salt): } x(t) &= A \operatorname{sgn}(\sin(2\pi f t + \phi)) = \frac{4A}{\pi} \sum_{k \text{ odd}} \frac{1}{k} \sin(2\pi k f t + \dots) \end{aligned}$$

The Fourier series explains what the Filtering and Oven spectra show: a square wave keeps strong odd harmonics ($1/k$), a triangle loses them quickly ($1/k^2$).

Parametric ingredients and their 1-D signal

A curve $\gamma(\theta) = (X(\theta), Y(\theta))$, $\theta \in [\theta_{\min}, \theta_{\max}]$, is sampled at 601 points for drawing. The 1-D signal uses the vertical component over the same parameter range, e.g. Noodles ($\theta \in [-12\pi, 12\pi]$):

$$x[n] = A \cos(-12\pi + 24\pi t_n + \phi) \quad (\text{i.e. } Y/5).$$

Frontend noise model (`ingredientNoise`)

Five tones above the ingredient's band (15.3, 18.7, 22.1, 26.9, 31.4 game Hz):

$$n(t) = 0.85 \nu \frac{1}{5} \sum_{i=1}^5 \sin(2\pi f_i t + 37.1 \phi_s i), \quad x_{\text{dirty}} = x - n.$$

Tones are used instead of a hashed pseudo-random sequence so the noise is genuinely spread above the cutoff instead of aliasing into one spike.

3.2 Backend ingredient synthesis and contamination

Backend signals are real audio: $F_s = 22\,050$ Hz, frame length 4096 (≈ 186 ms) (`backend/app/dsp/core.py`). `clean_signal` calls `instruments.synth` (additive partials with ADSR envelopes: piano, guitar, bell, flute, ..., plus mathematical voices for cheese, sugar, salt, bread, patty, carrot, cucumber, sauce, egg).

`contamination.corrupt(clean, difficulty  $d$ , seed)` uses a seeded RNG so the same session always reproduces the same dirty ingredient:

- number of layers = $1 + \text{rand}\{0, \dots, \min(2, \lfloor d \rfloor)\}$;
- each layer's amplitude $a = 0.08 + 0.10 d (0.6 + 0.8u)$, $u \sim U(0, 1)$;
- type chosen from **{white** (whole band), **hiss** (high-passed noise starting just above the ingredient's ideal cutoff $\times(1.1..1.6)$), **burst** (sparse clicks)};
- $x_{\text{dirty}} = x + \sum_{\ell} n_{\ell}$, and every contaminant's band $[f_{lo}, f_{hi}]$ is recorded.

4 Filtering

4.1 Server filter (authoritative)

`apply_filter_chain` (`backend/app/dsp/pipeline.py`) is an FFT-mask filter:

$$N = 2^{\lceil \log_2 L \rceil}, \quad X[k] = \text{FFT}_N\{x\}, \quad Y[k] = X[k] H(f_k), \quad f_k = \frac{kF_s}{N}, \quad y = \text{IFFT}_N\{Y\}[0..L-1].$$

The player's cutoff becomes a low-pass tool with a raised-cosine transition of width $w = \max(30, 0.15f_c)$ (`core.lowpass`):

$$H(f) = \begin{cases} 1 & f \leq f_c \\ 1 - \frac{1}{2}(1 - \cos \pi \frac{f-f_c}{w}) & f_c < f < f_c + w \\ 0 & f \geq f_c + w \end{cases}$$

(Graphic-EQ bands, high-pass, band-pass, notch and a harmonic “de-hum” comb are built the same way and multiplied: `chain.`)

4.2 Cleaning (prep) score — `metrics.prep_quality`

With magnitude spectra C (clean), D (dirty), F (filtered) and the set of noise bins $\mathcal{B}$ (inside any contaminant band):

$$\begin{aligned} \text{noise}_{\text{before}} &= \sum_{k \in \mathcal{B}} \max(0, D_k - C_k), & \text{noise}_{\text{after}} &= \sum_{k \in \mathcal{B}} \max(0, F_k - C_k), \\ \text{removal} &= \text{clip}\left(1 - \frac{\text{noise}_{\text{after}}}{\text{noise}_{\text{before}}}, 0, 1\right), & \text{preservation} &= \text{clip}\left(\frac{\sum_k \min(C_k, F_k)}{\sum_k C_k}, 0, 1\right), \\ \text{prep} &= 100(0.55 \text{removal} + 0.45 \text{preservation}). \end{aligned}$$

Flags: *over-filtered* if preservation < 0.55 ; *still dirty* if removal < 0.5 . `accept_ingredient` rejects prep < 45 or either flag. It is deliberately not a distance to the clean signal, which would reward doing nothing when the noise is quiet.

4.3 Frontend Filtering Lab (`routes/filtering.tsx`)

1. Input: the server's dirty/clean plots for the slot when a session exists, otherwise `getRecipeIngredientSamples` with noise 0.85 (dirty) and 0 (clean).
2. Live preview while dragging (cutoff $f_c \in [0, 900]$, ideal f^*): noise = dirty – clean; if $f_c < f^*$: signal gain $\max(0, 1 - \frac{f^* - f_c}{200})$, noise gain 0; else noise gain $(\frac{f_c - f^*}{900 - f^*})^{1.5}$.
3. *Apply*: sends a low-pass tool with that cutoff to the server and shows its filtered plot and prep score. Cleanliness = server prep score (offline: the similarity of filtered vs clean, or $97 - |f_c - f^*|/6$ before applying).
4. Pass mark: cleanliness ≥ 85 . *Advance* accepts the ingredient on the server, saves the filtered samples (`saveFilteredIngredient`) and records the stage accuracy as the running average of cleanliness over the washed ingredients.

5 Mixing

`computeMixedSignal` (`lib/pipeline.ts`)

For bowl ingredients $k = 1..K$ with samples x_k (the saved filtered samples if the ingredient was washed, otherwise its generated samples, noisy if washable):

$$m[n] = \text{normalize}_{0.95} \left(\frac{1}{\sqrt{K}} \sum_{k=1}^K x_k[n] \right), \quad K = 1 : m = x_1 \text{ (pass-through)}.$$

The $1/\sqrt{K}$ factor keeps the RMS of K uncorrelated signals comparable to one signal.

The mixing-bowl curve (`computeSuperpositionCurve`, `withMixCurve`)

If any bowl ingredient is parametric, the bowl is the geometric sum at a common normalised parameter $u \in [0, 1]$ (601 points):

$$X(u) = \sum_{k \in \text{param}} X_k(\theta_k(u)), \quad Y(u) = \sum_{k \in \text{param}} Y_k(\theta_k(u)) + \sum_{j \in 1-D} x_j(u).$$

The curve is attached as metadata (`curve`); the samples $m[n]$ are unchanged, so filtering still reaches the mix. Later stages transform the curve alongside the samples (`transformCurve`, `delayCurve`, `carryCurve`) and `pipelineSignalToPath` draws it.

Accuracy (`handleMix`, `routes/mixing.tsx`): $\text{mix} = 100 \cdot \frac{|\text{bowl} \cap \text{recipe}|}{|\text{recipe}|}$. The bowl's ingredient names are stored in the session and synced to the server.

Server (`bowl_signals`, `stage_mix`): recipe ingredients in the bowl use their filtered signal, extra catalogue ingredients are added clean, missing ones are absent; then $m = \frac{1}{\sqrt{K}} \sum_k \text{resize}(x_k, 4096)$.

6 Seasoning and marinating

6.1 Seasoning — `computeSeasonedSignal`

$$s[n] = A \cdot m((\alpha t_n) \bmod 1) \quad (\text{via } \text{timeScaleSamples}),$$

i.e. $y(t) = Ax(\alpha t)$ with the mix repeated periodically when $\alpha > 1$. No renormalisation, so A really changes the height. The frequency label becomes αf . The curve: $Y \mapsto Y_{\text{mid}} + A(Y - Y_{\text{mid}})$, parameter sped up by α with an x -offset per repeat. **Accuracy** (`routes/transform.tsx`):

$$\text{acc} = \max \left(0, 100 - 70 \left(\frac{|A-A^*|}{A^*} + \frac{|\alpha-\alpha^*|}{\alpha^*} \right) \right), \quad \text{pass at 85.}$$

Server: `stage_season` $y = Ax$; `stage_blend` $y[n] = x(\alpha n)$ by linear interpolation with zeros past the end (`core.time_scale`); length becomes $\lfloor L/\alpha \rfloor$.

6.2 Marinating — `computeMarinatedSignal`

With time s (seconds) and duration label $D = 3$:

$$d = \text{round} \left(\frac{s}{\max(1, D)} \cdot N \right), \quad r[n] = \begin{cases} m \left(\frac{n-d}{N-1} \right) & 0 \leq n-d \\ 0 & n < d \end{cases}, \quad r \leftarrow \text{normalize}_{0.95}(r).$$

The curve gets a flat lead-in at the centre line (`delayCurve`). **Accuracy**: $\max(0, 100 - 150 \frac{|s-s^*|}{s^*})$, pass at 85. **Server**: `stage_marinate` delays by sF_s samples after zero-padding; integer delays copy

samples, fractional delays use the shift theorem

$$Y[k] = X[k] e^{-j2\pi k d/N} \quad (\text{core.time_shift}).$$

7 Cooking — convolution

7.1 Impulse responses

Frontend kernels (`generateStaticCookingKernel`, 96 taps, $\sum_k |h[k]| = 1$):

$$h_{\text{grill}}[k] = e^{-k/16} (\tau_k + 0.15 \sin(1.85k) e^{-k/16}), \quad \tau_k \text{ taps } 1, .55, .38, .22, .12, .06 \text{ at } k = 0, 4, 11, 21, 35, 54$$

$$h_{\text{fry}}[k] = e^{-k/22} (0.45 \sin(12.99k + 3.14) + 0.35 \sin(31.42k + 1.2) + 0.2 \cos(57.17k))$$

$$h_{\text{bake}}[k] = e^{-((k-16)/14)^2} \quad (\text{Gaussian, a smooth low-pass})$$

$$h_{\text{boil}}[k] = e^{-k/26} (0.7 \sin(\pi k/7) + 0.3 \sin(\pi k/14)) \quad (\text{damped resonator})$$

Server impulse responses (`backend/app/dsp/systems.py`): grill = comb (delay 90, feedback 0.62, 7 taps), fry = short bright decay ($\tau = 130$), bake = long dark decay ($\tau = 700$, low-passed), boil = resonator $e^{-k/\tau} \sin(2\pi f_0 k/F_s)$ at 320 Hz, simmer = 48-tap moving average, sear = $[1, -1]$ (differencer), plus smoke and steam composites.

7.2 computeConvolvedSignal (`lib/pipeline.ts`)

Slider position $p \in [0, 100]$, $\tau = p/100$, shift $s = \text{round}(30\tau)$:

$$c[n] = \sum_{k=0}^{95} x((n - k - s) \bmod N) h[k] \quad (\text{circular convolution}),$$

$$y[n] = (1 - w) x[n] + w c[n], \quad w = \min(1, 1.15\tau),$$

plus a small crackle $0.04\tau \sin(27.13n + 7.4)$ for frying, then normalised to 0.95. **Accuracy** (`routes/cooking.tsx`): 50 if the recipe's method is chosen (else 15) + $\min(50, p/2)$. The target cooked signal uses the recipe's method at $p = 100$.

Server (`stage_cook`, `apply_system`): linear convolution $y = x * h$ (SciPy, direct or FFT), truncated to the input length, rescaled to 0.95 only if it clips. Several appliances are one system: `cascade` $h = h_1 * h_2 * \dots$.

8 Precision Oven (`frontend/src/lib/precision-oven-dsp.ts`)

8.1 The problem: a burnt overtone (`frontend/src/lib/delivery.ts`)

Cooking leaves a recipe-specific tone in the dish. With f_0 the fundamental of the reference dish (the strongest DFT bin, `dishFundamental`) and the recipe's profile (ρ, a_d) (`DELIVERY_PROFILES`):

$$f_d = \text{clip}(\rho f_0, 10, 28), \quad x_{\text{served}}[n] = x_{\text{cooked}}[n] + a_d \max |x_{\text{cooked}}| \sin\left(2\pi f_d \frac{n}{N-1}\right) \quad (\text{servedDish}).$$

ρ is chosen so f_d is an integer inside the notch slider's range and at least 1 Hz from every harmonic of f_0 (e.g. burger $f_0 = 4$, $f_d = 18$ between harmonics 16 and 20); a_d rises with the tier (0.8 for Burger and Sandwich, 0.9 for Cake and Noodles, 1.0 for Chicken Fry). Check Dish shows and plays x_{served} , the oven receives it, and serving without the oven keeps it.

8.2 Maximum frequency and the Nyquist rate

findMaxSignalFrequency: resample the dish to 128 points, apply a Hann window $w[n] = \frac{1}{2}(1 - \cos \frac{2\pi n}{127})$, FFT, and take the first bin k at which the cumulative energy $\sum_{i=1}^k |X_i|^2$ reaches 95 % of the total (DC excluded); with a 128-point window over one second each bin is 1 game Hz wide, so $f_{\max} = \text{clip}(k, 2, 32)$ game Hz. Minimum safe rate $= 2f_{\max}$.

8.3 Sampling and aliasing demonstration — **analyzeSamplingAndAliasing**

1. Samples $x_m = x(m/f_s)$ for $m = 0..[f_s] - 1$ over the one-second window.
2. Periodic sinc reconstruction over three periods: $\hat{x}(t) = \sum_{p=-1}^1 \sum_m x_m \text{sinc}(f_s(t - \frac{m}{f_s} - p))$.
3. Folding of the five strongest harmonics when $f > f_s/2$: $f_{\text{alias}} = |f - \text{round}(f/f_s)f_s|$ (reflected again if still above $f_s/2$).

8.4 Spectrum used for tuning — **computeDiscreteSignalFFT**

$N_{\text{FFT}} = \max(64, 2^{\lceil \log_2 f_s \rceil})$, $x[m] = x(m/f_s)$ ($m < f_s$), 0 otherwise (zero padding), $f_k = \frac{k f_s}{N_{\text{FFT}}}$.

No window is applied (the edited spectrum is inverted back). Displayed magnitude $|X_k| \cdot 2/f_s$, normalised by 1.25 max. Radix-2 Cooley–Tukey FFT with bit reversal (**cooleyTukeyRadix2**). Bins are classed as Base Warmth $[0, 6)$, Crumb $[6, 16)$, Top Crisp $[16, 32]$ game Hz (**getBandAvailability** marks a band unavailable/partial when $f_s/2$ is below it).

8.5 Equaliser — **tuneOvenFrequencies**

$$X'_k = X_k \cdot G_{\text{band}(f_k)} \cdot C(f_k) \cdot Q(f_k), \quad C(f) = \begin{cases} 1 & f \leq f_c \\ \max(0.01, e^{-((f-f_c)/4)^2}) & f > f_c \end{cases},$$

$$Q(f) = 1 - \frac{0.98}{1 + ((f - f_n)/1.5)^2} \quad \text{for } |f - f_n| \leq 3 \text{ (notch on),}$$

$$X'_{N-k} = \overline{X'_k} \quad (\text{Hermitian symmetry, so the result stays real}).$$

The same gain $G \cdot C \cdot Q$ is **ovenGainAt**(f); the notch rejects 98 % at its centre (an earlier 0.88 left 12 % of the overtone in the dish).

8.6 Reconstruction — **computeSignalIFFT**

IFFT, keep the first $N_s = [f_s]$ samples, then band-limited periodic interpolation with the Dirichlet kernel back to 401 points:

$$\hat{x}(t) = \sum_{m=0}^{N_s-1} x'_m D_{N_s}\left(t - \frac{m}{N_s}\right), \quad D_N(\tau) = \begin{cases} \frac{\sin(\pi N \tau)}{N \sin(\pi \tau)} & N \text{ odd} \\ \frac{\sin(\pi N \tau)}{N \tan(\pi \tau)} & N \text{ even} \end{cases}$$

keeping the dish's own amplitude (no rescaling, so a level mistake stays visible). This rebuild from the f_s samples is what the sampling stage shows and plays, so aliasing is heard when $f_s < 2f_{\max}$.

The plated dish (**applyOvenToDish**): once $f_s \geq 2f_{\max}$ is verified, sampling is lossless, so the served dish is the equaliser applied to the full-resolution dish: a direct DFT of one period (400 samples, bin $k = k$ game Hz), $X_k \leftarrow X_k \text{ovenGainAt}(k)$ for $k \leq f_s/2$, unchanged above, then the inverse DFT. (Rebuilding it from the f_s samples lost everything above $f_s/2$, and $f_{\max}$ only

holds 95 % of the energy, so even a perfect run sounded different.) With untouched controls it returns the dish exactly (`frontend/tests/precision-oven-fft.test.ts`); with the notch on f_d it returns the dish *without* the overtone. Served straight from Check Dish (overtone kept) a dish matches the target at 45–61 %; after a perfect oven and cart it matches the cooked dish at 99–100 % (`frontend/tests/delivery.test.ts`, `frontend/tests/playground.test.ts`).

8.7 Oven score

$$\text{spectrum match} = \text{clip}\left(100\left(1 - \frac{\sum_k |c_k - t_k|}{1.5 \sum_k t_k}\right), 0, 100\right) \quad (\text{computeSpectrumSimilarity})$$

$$\text{waveform fidelity} = \text{computeSignalSimilarity}(\hat{x}, \text{target}) \quad (\text{Section 10})$$

$$\text{efficiency} = \begin{cases} 100 & f_s \leq 1.25 \cdot 2f_{\max} \\ 100\left(1 - \frac{f_s - 2.5f_{\max}}{64 - 2.5f_{\max}}\right)^+ & \text{otherwise} \end{cases}$$

$$\text{oven} = 0.40 \text{ spectrum} + 0.50 \text{ fidelity} + 0.10 \text{ efficiency}.$$

The unlock step rejects $f_s < 2f_{\max}$ and $f_s > 64$. *Plate* saves the reconstruction as the **delivered** stage (the dish that goes on to the cart), records the oven score, and stores the equaliser *settings* ($f_0, G_{\text{low}}, G_{\text{mid}}, G_{\text{high}}, f_c, f_n$, notch on) for the server.

Server (`apply_oven`, `backend/app/delivery.py`): the same overtone is added to the server’s dish at ρf_0^{srv} (its own fundamental in real Hz), then the same $G \cdot C \cdot Q$ mask is applied with each FFT bin mapped to game Hz as $g = f / f_0^{\text{srv}} \cdot f_0$, so “the notch sits on the overtone” means the same on both sides.

9 System Delivery — the z-plane (`frontend/src/lib/z-system.ts`)

Preset	$H(z)$	Poles / zeros / gain
1st-order low-pass	$\frac{(1-r)z}{z-r};$ $y[n] = (1-r)x[n] + ry[n-1]$	pole r , zero 0, $g = 1 - r$ (unity DC gain)
2nd-order resonator	$\frac{z^2}{(z - re^{j\omega_0})(z - re^{-j\omega_0})}$	conjugate poles, double zero at 0, $g = 1$; $y[n] = x[n] + 2r \cos \omega_0 y[n-1] - r^2 y[n-2]$
Moving average	$\frac{1}{6} \frac{1 - z^{-6}}{1 - z^{-1}}$	zeros $e^{j2\pi k/6}$, $k = 1..5$; five poles at 0; $g = 1/6$ (FIR)
Notch	$g \frac{(z - e^{j\omega_0})(z - e^{-j\omega_0})}{(z - 0.85e^{j\omega_0})(z - 0.85e^{-j\omega_0})}$	zeros on the unit circle, poles at radius 0.85; g makes $ H = 1$ at DC or Nyquist, whichever is farther from ω_0 (with $g = 1$ the dish itself was amplified about $\times 1.17$)

Frequency response and filtering

- Magnitude on the unit circle, geometrically (`SystemResponsePlotter.tsx`): $|H(e^{j\omega})| = g \frac{\prod_i |e^{j\omega} - z_i|}{\prod_j |e^{j\omega} - p_j|}$ (clipped to 4), phase = $\sum \arg(e^{j\omega} - z_i) - \sum \arg(e^{j\omega} - p_j)$, 128 points on $[0, \pi]$.
- Dish spectrum: 64-point DFT of the dish; bin $k \leq 32$ sits at $\omega_k = 2\pi k/64$ and is multiplied by $|H(e^{j\omega_k})|$ (an unstable cart shows $\times 5$).
- Actual filtering (`applySystem`): expand $\prod (z - p_j)$ and $g \prod (z - z_i)$ into real coefficients a ,

b (`expandRoots`), then run the difference equation $y[n] = \sum_k b_k x[n-k] - \sum_{k \geq 1} a_k y[n-k]$. This plays as “Listen equalized dish”.

The problem: road vibration (`deliverOnCart`, `frontend/src/lib/delivery.ts`)

The road adds a tone at f_r (recipe profile, defined at the true rate $F_d = 8000$ Hz):

$$\omega_r = \frac{2\pi f_r}{F_d}, \quad v[n] = x[n] + a_r \max |x| \cos(\omega_r n), \quad y = H\{v\} \text{ (steady state).}$$

The dish loops and every road frequency is a whole number of cycles per window, so the cart’s *steady-state* response is used (`cartServe`): the filter runs over three warm-up periods first and the fourth is served. Starting from rest left a start-up transient worth up to the dish’s whole peak. The output keeps the dish’s level; only a runaway output (above $1.5 \max |x|$, from an unstable cart or a resonance on the road) is scaled back to the dish’s peak, so its ringing, not its size, is judged. Score = `computeSignalSimilarity`(served, dish before the road). A zero on the unit circle at ω_r removes the tone and leaves the dish; a low-pass pole damps it but also dulls the dish; poles outside $|z| = 1$ make y grow without bound.

Aliased sensing (`sensedRoadOmega`): the sensor samples at f_s and reports $f_a = |f_r \bmod f_s|$ folded into $[0, f_s/2]$, i.e. $\hat{\omega} = 2\pi f_a / f_s$, drawn as the orange marker. At 8 kHz $\hat{\omega} = \omega_r$; at 1.2 kHz it is wrong (burger: $2.04 \rightarrow 1.05$), so a notch aimed at the marker misses the real vibration (server dish score 9–23 instead of 93–98).

Server (`deliver_on_cart`, `backend/app/delivery.py`): the same tone at the same ω_r (rad/sample), the cart’s coefficients from the synched (`preset, r, \omega_0`), and `scipy.signal.lfilter` for the difference equation.

10 Similarity metrics

10.1 Frontend (`computeSignalSimilarity`, `frontend/src/lib/dsp.ts`)

The player’s samples are resampled to the target length, then

$$r_{xy} = \frac{\sum (x - \bar{x})(y - \bar{y})}{\sqrt{\sum (x - \bar{x})^2 \sum (y - \bar{y})^2}}, \quad \text{NRMSE}_{\text{sim}} = \text{clip}\left(1 - \frac{\sqrt{\frac{1}{N} \sum (x - y)^2}}{\max y - \min y}, 0, 1\right),$$

$$\text{similarity} = 100(0.5 \max(0, r_{xy}) + 0.5 \text{NRMSE}_{\text{sim}}).$$

Correlation rewards the right *shape*; NRMSE rewards the right *level*.

10.2 Server dish score (`metrics.dish_metrics`)

For the served dish both signals are scaled by the same factor (the reference’s peak to 0.9), so a level error such as over-seasoning counts; the station sub-scores compare shapes (each normalised to peak 0.9). Then

$$\text{SNR} = 10 \log_{10} \frac{\sum t^2}{\sum (t - p)^2} \text{ dB}, \quad \text{corr} = \max_{\ell} \frac{(t \star p)[\ell]}{\|t\| \|p\|} \text{ (peak of the cross-correlation),}$$

$$\text{spec} = \frac{\langle T_{\text{dB}}, P_{\text{dB}} \rangle}{\|T_{\text{dB}}\| \|P_{\text{dB}}\|}, \quad T_{\text{dB}} = \max\left(20 \log_{10} \frac{|T|}{\max |T|}, -40\right) + 40,$$

with P_{dB} measured against the *target’s* peak as well, so only the 40 dB that make the dish are compared (an absolute -80 dB floor made every dish score 0.98–1.00).

$$\text{dish} = 100\left(0.40 \text{clip} \frac{\text{spec} - 0.60}{0.38} + 0.35 \text{clip}(\text{corr}) + 0.25 \text{clip} \frac{\text{SNR} + 5}{30}\right) \text{ (each clip to } [0, 1]).$$

Perfect dishes measure $\text{spec} = 0.984\text{--}0.996$; broken ones $0.5\text{--}0.9$.

11 Scoring, step by step

11.1 During the run: stage accuracies

Each station calls `recordStageAccuracy` (`lib/recipes.ts`), which clamps to $[0, 100]$ and stores it on the run session: filtering, mixing, seasoning, marinating, cooking, delivery (oven), system (cart). Dials are stored too and pushed to the server by `syncSessionParamsToBackend`. The payload fields are `seasoning`, `frequency`, `blend`, `marinate`, `appliances`, `bowl`, the oven settings `oven_f0`, `oven_gains`, `oven_cutoff`, `oven_notch`, `oven_notch_on`, `oven_fs`, and the cart settings `system_preset`, `system_pole_radius`, `system_omega` and `system_sampling_hz`. No score is sent: the server computes every one.

11.2 The reference dish

Frontend `getIdealDishSignal`: clean mix of all recipe ingredients (`computeCleanMixedSignal`) $\rightarrow$ seasoning with $(A^*, \alpha^*) \rightarrow$ marinating with $s^* \rightarrow$ convolution with the recipe method at $p = 100$.
Server `judge`: `run_pipeline(clean ingredient signals, reference_params(recipe))` — the same function that builds the player's dish.

11.3 Server judgement — `gameplay.judge`

1. Rebuild the player's dish: regenerate each ingredient from its seed, apply the stored filters, keep only the bowl, run `run_pipeline` with the stored params: $\text{mix} \rightarrow \text{season } (A) \rightarrow \text{blend } (\alpha) \rightarrow \text{marinate } (s) \rightarrow \text{cook (appliances)}$. (The server pipeline still has optional AM and decimation stages; no recipe uses them, so they pass the signal through.)
2. Finish it (`finish_dish`): add the burnt overtone; if the oven was used apply its equaliser (oven sub-score = dish score after the oven); if the cart was used add the road vibration and filter with $H(z)$ (cart sub-score = dish score after the cart). Serving from Check Dish skips both, so the overtone stays.
3. $m = \text{dish_metrics}(\text{reference cooked dish, served dish})$ on a common scale.
4. Prep average $\bar{P}$ over washable/contaminated ingredients (all ingredients if none).
5. Sub-scores: filtering = $\bar{P}$; mixing = dish score of the two mixes; transformation = dish score of the two marinated signals; cooking = dish score of (player's pre-cooking signal through the recipe's appliances) vs (through the player's).
6. $\text{raw} = m_{\text{score}}(0.72 + 0.28 \bar{P}/100)$: washing scales the dish score, so clean prep cannot carry a ruined dish (with a perfect dish this equals $0.72 m + 0.28 \bar{P}$). The finishing stations count through m , because they change the served dish itself.
7. Tolerance curve: $\text{score} = 100 (\text{raw}/100)^\gamma$, $\gamma = 1/\max(0.3, \text{tolerance})$ (tolerance 0.9 for Chicken Fry up to 1.15 for Sandwich: harder recipes are graded more strictly).
8. Stars: $\geq 92 \rightarrow 5$, $\geq 80 \rightarrow 4$, $\geq 66 \rightarrow 3$, $\geq 50 \rightarrow 2$, $\geq 32 \rightarrow 1$, else 0.
9. Notes (`_diagnose`): named mistakes from the parameter errors (over/under-seasoned, blended, marinated, flagged once a dial is more than 5 % off its target; wrong appliance chain; over-filtered or still-dirty ingredients) and the finishing stations (how much of the burnt overtone the oven kept, how the cart scaled the road vibration and the dish, an unstable cart, a notch aimed at the aliased sensor reading).

11.4 Submission (exactly once)

`submitRunToBackend` (`lib/recipes.ts`) syncs the params, then `POST /sessions/{id}/submit`; concurrent callers share one in-flight request. The server claims the session atomically (`UPDATE ... WHERE status != 'served'`); a second submit gets 409; if judging fails the claim is released. A served session can no longer be edited.

11.5 What the score screen shows (`routes/score.tsx`)

The player dish is the served dish: the cart's output (`served` stage), else the oven's output (`delivered`), else the cooked dish *with* the burnt overtone. The stage average is the mean of filtering, mixing, seasoning, marinating and cooking.

```

similarity = computeSignalSimilarity(player dish, reference dish)
local = 0.5 similarity + 0.5 stageAvg
score100 = server score if available, otherwise local
base = round(10 · score100 · μdiff),    μ ∈ {0.8, 1.0, 1.25, 1.5}
time bonus = round(min(2 · seconds left, 300) · μdiff)
total = base + time bonus.

```

A stage with no recorded accuracy counts as 0 (except filtering on a recipe with nothing to wash, which counts 100). Stars use the server's stars or the same thresholds on `score100`. The total, stars and similarity are stored on the session; the Complete page (`routes/complete.tsx`) only displays them.

12 Points, ranks, tiers and leaderboards

12.1 Career points and ranks (`backend/app/gameplay.py`)

- **award**: if the new score beats the chef's best on that recipe, `points = round(score × tier × 1.5)` is added; otherwise 0. Runs on a tier the chef has not unlocked earn 0 and do not count for unlocking.
- **rank_for**: Dishwasher (0), Prep Cook (250), Line Cook (600), Sous Chef (1100), Head Chef (1800), Signal Master (2600).
- **maybe_unlock**: when the chef has cleared `min(2, #recipes)` distinct recipes of their current tier with score ≥ 60 , the next tier opens.

12.2 Server leaderboards (`backend/app/routers/leaderboard.py`)

GET /leaderboard?recipe_id=&difficulty=&limit=

This season's attempts ordered by overall score (then time), de-duplicated to each chef's best, then cut to `limit` (at most 500). With a difficulty, only attempts of that difficulty; runs from before difficulty was recorded count as Easy.

GET /leaderboard/global

Chefs who have served a dish, ordered by career points, then best overall score this season; with rank title and dishes served.

GET /stats

Players, sessions started, and for this season's runs on active recipes: dishes served, average score, and per recipe plays, average, best and five-star rate.

Seasons

Every attempt records the scoring rules that judged it (`SCORING_VERSION`). Only runs of the current version are ranked, count as personal bests and towards tier unlocks, or enter the statistics; older runs stay stored, and the points and tiers they earned are kept.

12.3 Leaderboard page (`frontend/src/routes/leaderboard.tsx`)

Each board shows the **top 5** rows, plus the signed-in chef's own row when it is ranked lower (`visibleRows`); *See all* fetches every row (up to 500). A request id guards against a slow earlier request overwriting a newer filter's rows, and a submission still in flight is awaited before the board reloads, so a run just served appears at once.

12.4 On-device leaderboard (`frontend/src/lib/leaderboard.ts`)

The score screen calls `addLeaderboardEntry` with id `run-<recipe>-<difficulty>-<start>`: one entry per chef, recipe and difficulty, keeping the higher score. `getLeaderboard` filters by recipe and difficulty and sorts by score. The leaderboard page shows the server's rows when it has any, otherwise these; it also re-submits a finished run that never reached the server (backend offline) and shows a retry notice.

13 Timer, pause and sound

- **Timer** (`useRecipeTimer`, `lib/recipes.ts`): Easy 7:00, Medium 5:00, Hard 3:10, MasterChef 2:00 (`DIFFICULTY_CONFIGS`; the multipliers 0.8/1.0/1.25/1.5 mirror `DIFFICULTIES` in `backend/app/gameplay.py`). At zero the expiry dialog blocks the run.
- **Start countdown** (`components/game/StationCountdown.tsx`): starting a recipe shows 3–2–1–GO with a short tone each; the timer starts when it ends (`countdownPending`).
- **Pause** (`pauseRecipeRun`, `resumeRecipeRun`, `components/game/PauseModal.tsx`): freezes the timer (the time left is stored) and pauses every playing signal; the pause menu offers resume, restart, change difficulty, sound on/off and volume, and the theme.
- **Sound settings** (`lib/sound.ts`): on/off and volume, applied to signal playback and to the sound effects.
- **Sound effects** (`lib/sfx.ts`): one delegated click listener plays a short Web Audio tick for every button; each cooking method has one looping recording (grill, fry, bake, boil, in `frontend/sounds`) that plays while the convolution slider is held and stops on release, on leaving the page, or after a short keyboard nudge.

The server times the overall score's bonus on its own clock from session creation to submission, so pause time is not subtracted there (the score screen shows the server's total).

14 Signal Playground (`frontend/src/lib/playground.ts`)

The playground runs every station on any ingredient or oscillator by calling the stations' own functions, so a signal looks and sounds exactly as in a run:

- `sourceSamples`, `washSource`, `mixSources`, `seasonSource`, `marinateSource`, `cookSource`: the ingredient models, `washFilterPreview`, `superposeSamples`, `computeSeasonedSignal`, `computeMarinatedSignal`, `computeConvolvedSignal`.

- **bakeInOven:** `addBurntOvertone` at a frequency between the signal's harmonics (`overtoneBetweenHarmonics`), `computeDiscreteSignalFFT` at f_s , `tuneOvenFrequencies`; the output is `applyOvenToDish` at $f_s \geq 2f_{\max}$, else the aliased rebuild `computeSignalIFFT`.
- **deliverOnPlaygroundCart:** `addRoadTone`, `cartServe` with `systemPolesZeros`, $|H|$ at the road and at DC (`gainAt`), stability, and the sensor reading `aliasedOmega`.
- **tasteCompare:** the live match (`computeSignalSimilarity`) and the server's dish score (common scale, SNR, best-lag correlation, 40 dB spectra) on any two signals.
- **runFullChain:** `computeCleanMixedSignal` $\rightarrow$ `season` $\rightarrow$ `marinate` $\rightarrow$ `cook` $\rightarrow$ `servedDish` $\rightarrow$ oven notch on `ovenDefectHz` $\rightarrow$ cart notch on `roadOmega`, compared with `getIdealDishSignal`; at `targetDials` it reproduces the reference dish (99–100 %, tested for every recipe).

The lab screens are `routes/labs.tsx` and `components/playground/*`.

15 Security and validation

- Passwords: `scrypt` ($N = 2^{14}$, $r = 8$, $p = 1$, 16-byte salt), constant-time comparison (`backend/app/security.py`). A chef created before passwords existed gets one on first sign-in.
- Every session endpoint checks the token owns the session (`owned_session`).
- DSP endpoints bound every input (sample rate 1–96 kHz, $\leq 200\,000$ finite samples, ≤ 4096 -tap custom impulse).
- The SPA file route only serves files inside `frontend/dist`; CORS allows only the frontend's origins.
- The server never trusts client audio: it regenerates ingredients from the seed and rebuilds the dish, including the finishing stations from their settings, so no score is taken from the client.

16 Function index

Function	File	Mathematical role
Frontend		
<code>evaluate*Wave, generate*Samples</code>	<code>lib/signals.ts</code>	Ingredient signal models
<code>ingredientNoise</code>	<code>lib/signals.ts</code>	Additive multi-tone noise
<code>computeSuperpositionCurve</code>	<code>lib/signals.ts</code>	2-D geometric superposition
<code>getRecipeIngredientSamples</code>	<code>lib/pipeline.ts</code>	Ingredient $\rightarrow$ 401 samples
<code>sampleAt, resampleSignal, normalizeSamples</code>	<code>lib/pipeline.ts</code>	Interpolation, resampling, peak limiting
<code>computeMixedSignal</code>	<code>lib/pipeline.ts</code>	Superposition with $1/\sqrt{K}$
<code>withMixCurve</code>	<code>lib/pipeline.ts</code>	Attach bowl curve
<code>computeSeasonedSignal, timeScaleSamples</code>	<code>lib/pipeline.ts</code>	$Ax(\alpha t)$
<code>computeMarinatedSignal, delayCurve</code>	<code>lib/pipeline.ts</code>	$x(t - t_0)$
<code>computeConvolvedSignal</code>	<code>lib/pipeline.ts</code>	Circular convolution, wet/dry
<code>carryCurve, transformCurve, pipelineSignalToPath</code>	<code>lib/pipeline.ts</code>	Curve follows each stage (display)
<code>getIdealDishSignal, computeCleanMixedSignal</code>	<code>lib/pipeline.ts</code>	Reference dish

Function	File	Mathematical role
generateStaticCookingKernel	lib/cooking-audio.ts	Impulse responses $h[n]$
fft, applyLowPassFilter, convolve	lib/dsp.ts	FFT, filtering, convolution
normalizedCrossCorrelation, normalizedRootMeanSquareError, computeSignalSimilarity	lib/dsp.ts	Similarity
findMaxSignalFrequency	lib/precision-oven-dsp.ts	95 %-energy bandwidth
analyzeSamplingAndAliasing	lib/precision-oven-dsp.ts	Sampling, sinc reconstruction, folding
computeDiscreteSignalFFT, cooleyTukeyRadix2	lib/precision-oven-dsp.ts	Zero-padded DFT / FFT
tuneOvenFrequencies	lib/precision-oven-dsp.ts	Band gains, cutoff, notch
computeSignalIFFT	lib/precision-oven-dsp.ts	IFFT + Dirichlet reconstruction
computeSpectrumSimilarity	lib/precision-oven-dsp.ts	Spectral match
systemPolesZeros, applySystem	lib/z-system.ts	$H(z)$, difference equation
servedDish, dishFundamental, ovenDefectHz	lib/delivery.ts	Burnt overtone
deliverOnCart, cartServe, withRoadVibration, sensedRoadOmega	lib/delivery.ts	Road vibration, steady-state cart, aliased sensing
ovenGainAt, applyOvenToDish	lib/precision-oven-dsp.ts	Oven gain, the plated dish
gainAt	lib/z-system.ts	$ H(e^{j\omega}) $ at one frequency
bakeInOven, deliverOnPlaygroundCart, tasteCompare, runFullChain	lib/playground.ts	Playground labs
pauseRecipeRun, resumeRecipeRun, useRecipeTimer	lib/recipes.ts	Timer and pause
startCookingSfx, stopCookingSfx, installButtonClickSfx	lib/sfx.ts	Sound effects
SignalAudioPlayer	lib/audio.ts	Sonification
recordStageAccuracy, syncSessionParamsToBackend, submitRunToBackend	lib/recipes.ts	Accuracies, sync, submission
addLeaderboardEntry, getLeaderboard	lib/leaderboard.ts	Local leaderboard
Backend		
fft, ifft, cooley_tukey_fft	dsp/core.py	Transforms
lowpass, highpass, bandpass, notch, equalizer, chain	dsp/core.py	Frequency responses

Function	File	Mathematical role
build_mask, apply_mask, filter_signal	dsp/core.py	FFT-mask filtering
time_scale, time_shift	dsp/core.py	$x(\alpha t)$, $x(t - t_0)$
convolve, apply_system, cascade, frequency_response	dsp/core.py	LTI systems
am_modulate, decimate	dsp/core.py	AM, down-sampling (unused by the current recipes)
corrupt	dsp/contamination.py	Seeded noise
synth	dsp/instruments.py	Ingredient audio
ir_comb, ir_decay, ir_resonator, ir_moving_average, ir_difference	dsp/systems.py	Appliance $h[n]$
run_pipeline, reference_params, apply_filter_chain	dsp/pipeline.py	The canonical dish pipeline
prep_quality, dish_metrics, snr_db, spectral_similarity, stars	dsp/metrics.py	Scores
bowl_signals, compute_stages, finish_dish, judge, award, maybe_unlock	gameplay.py	Judging and progression
add_burnt_overtone, apply_oven, add_road_vibration, deliver_on_cart	delivery.py	Finishing stations on the server's dish